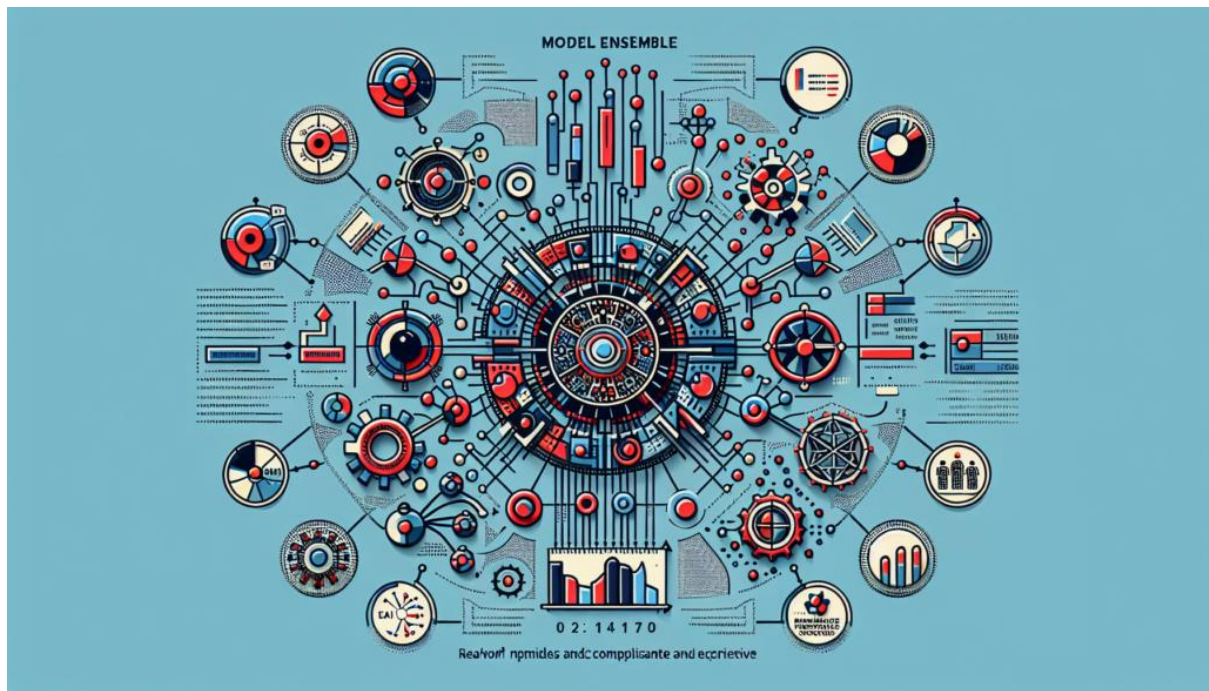




Autores: Rubén Pérez González, Miguel Ruiz Merino

Fecha: 18/02/2026

Asignatura: Sistemas de Aprendizaje Automático (IA)

PARTE 1 – Exploración inicial

```
--- 1.1 VISTA PREVIA ---  


|   | survived | pclass | sex    | age  | sibsp | parch | fare    | embarked | class | who   | adult_male | deck | embark_town | alive | alone |
|---|----------|--------|--------|------|-------|-------|---------|----------|-------|-------|------------|------|-------------|-------|-------|
| 0 | 0        | 3      | male   | 22.0 | 1     | 0     | 7.2500  | S        | Third | man   | True       | NaN  | Southampton | no    | False |
| 1 | 1        | 1      | female | 38.0 | 1     | 0     | 71.2833 | C        | First | woman | False      | C    | Cherbourg   | yes   | False |
| 2 | 1        | 3      | female | 26.0 | 0     | 0     | 7.9250  | S        | Third | woman | False      | NaN  | Southampton | yes   | True  |
| 3 | 1        | 1      | female | 35.0 | 1     | 0     | 53.1000 | S        | First | woman | False      | C    | Southampton | yes   | False |
| 4 | 0        | 3      | male   | 35.0 | 0     | 0     | 8.0500  | S        | Third | man   | True       | NaN  | Southampton | no    | True  |

  
--- 1.2 INFORMACIÓN DEL DATASET ---  
<class 'pandas.core.frame.DataFrame'  
RangeIndex: 891 entries, 0 to 890  
Data columns (total 15 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   survived         891 non-null    int64  
1   pclass           891 non-null    int64  
2   sex              891 non-null    object  
3   age              714 non-null    float64  
4   sibsp            891 non-null    int64  
5   parch            891 non-null    int64  
6   fare             891 non-null    float64  
7   embarked         889 non-null    object  
8   class            891 non-null    category  
9   who              891 non-null    object  
10  adult_male       891 non-null    bool  
11  deck             203 non-null    category  
12  embark_town      889 non-null    object  
13  alive            891 non-null    object  
14  alone            891 non-null    bool  
dtypes: bool(2), category(2), float64(2), int64(4), object(5)  
memory usage: 80.7+ KB  
None  
...  
survived  
0    0.616162  
1    0.383838  
Name: proportion, dtype: float64
```

1. ¿Cuántas filas y columnas tiene?

El dataset contiene 891 filas registros de pasajeros y 15 columnas variables.

2. ¿Qué variable será el target?

La variable objetivo a predecir es survived (0 = No sobrevivió, 1 = Sobrevivió).

3. ¿Hay valores nulos?

Sí. deck: Tiene solo 203 valores no nulos (faltan 688) age: Tiene 714 valores no nulos (faltan 177) embarked: Faltan 2 valores.

4. ¿Qué variables parecen más relevantes?

sex: Debido a la norma "mujeres y niños primero".

pclass: El estatus socioeconómico influyó en el acceso a los botes.

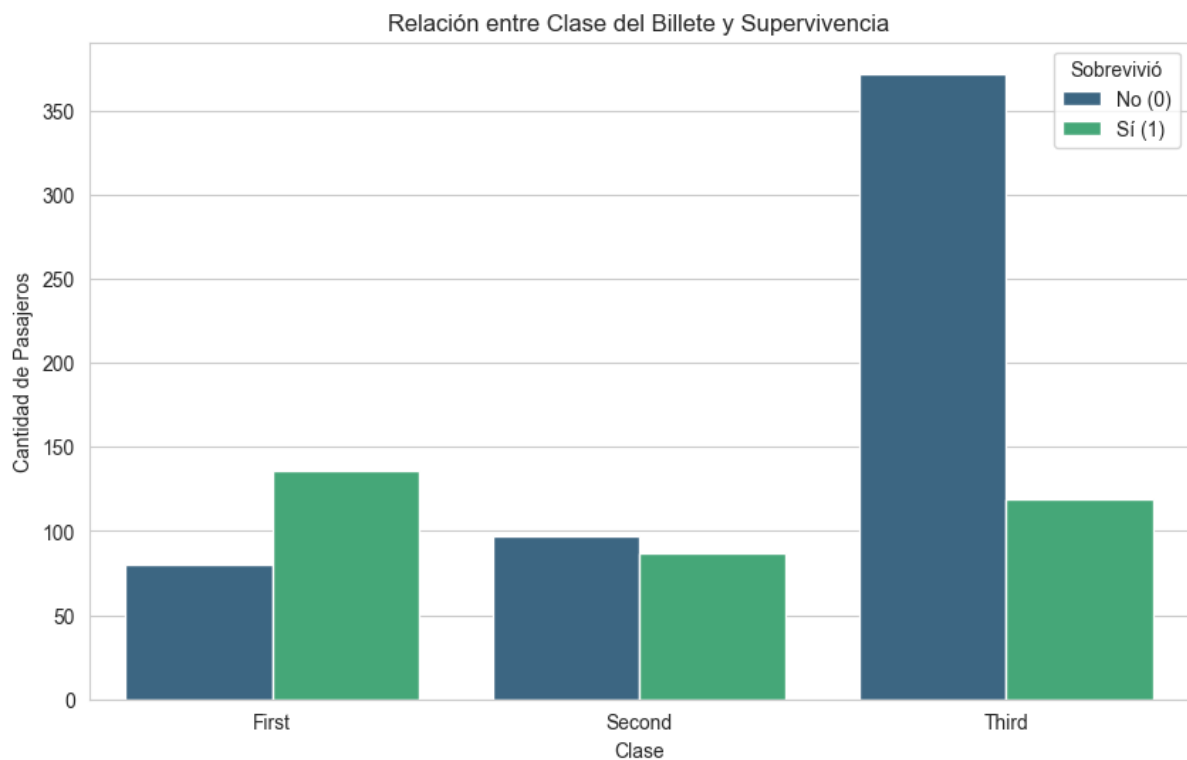
age: Prioridad a niños y ancianos.

fare: Relacionado directamente con la clase.

5. ¿El dataset está balanceado?

No, no sobrevivió 61.6% y sobrevivió 38.4%. Hay una clase mayoritaria, los fallecidos, por lo que está desbalanceado.

6. Gráfica



La gráfica de barras muestra la Supervivencia por Clase de Billeto.

En la 1ª Clase, la barra verde supera a la azul, indicando una alta tasa de supervivencia. En la 3ª Clase, la barra azul es mucho más alta que la verde, indicando una mortalidad masiva. Esto confirma visualmente que la clase social fue determinante.

PARTE 2 – Preprocesamiento

2.1 Exploración y selección de variables

1. ¿Qué variables son numéricas y cuáles categóricas?

Numéricas: age, sibsp (hermanos/cónyuges), parch (padres/hijos), fare.

Categóricas: survived, pclass, sex, embarked, class, who, adult_male, deck, embark_town, alive, alone.

2. ¿Hay columnas que parecen duplicadas o redundantes?

Sí, pclass y class representan lo mismo (código vs texto). embarked y embark_town representan lo mismo (código vs ciudad). survived y alive son idénticas (target). who y adult_male son redundantes teniendo sex y age.

3. ¿Qué columnas tienen demasiados valores nulos?

La columna deck (cubierta) tiene más del 77% de valores nulos.

4. ¿Eliminarías alguna variable? ¿Por qué?

Sí, eliminaría deck porque faltan demasiados datos para imputarlos fiablemente y eliminaría las redundantes (class, embark_town, alive, who) para evitar ruido y multicolinealidad.

5. ¿Crees que todas las variables aportan información relevante?

No. Variables redundantes solo duplican información. Además, si hubiera variables de identificación, tampoco aportarían capacidad de generalización.

6. ¿Es mejor eliminar variables con muchos nulos o intentar imputarlas?

Si el porcentaje de nulos es muy alto como en deck, >70%, es mejor eliminar la variable, ya que imputar tanto inventaría demasiada información. Si son pocos nulos como en age, es mejor imputar.

7. ¿Eliminar variables puede reducir varianza?

Sí. Al eliminar variables irrelevantes o ruidosas, evitamos que el modelo aprenda patrones espurios, reduciendo el riesgo de sobreajuste y, por tanto, la varianza.

2.2 Tratamiento de valores nulos

```
Nulos restantes tras limpieza:  
survived      0  
pclass        0  
sex           0  
age           0  
sibsp         0  
parch         0  
fare          0  
embarked      0  
alone         0  
dtype: int64
```

1. ¿Por qué usar mediana en lugar de media para la edad?

La mediana es más robusta a valores atípicos outliers. Si hubiera unos pocos pasajeros muy ancianos, la media subiría artificialmente, mientras que la mediana representa mejor al "pasajero típico".

2. ¿Qué ocurre si no tratamos los valores nulos?

La mayoría de los algoritmos de Árboles, Random Forest, SVM fallarán y darán error al intentar entrenar, ya que no pueden realizar cálculos matemáticos con valores NaN.

3. ¿Qué sesgo podría introducir la imputación?

Asumimos que los datos faltantes se comportan igual que los presentes Missing Completely At Random. Si, por ejemplo, los pasajeros de 3ª clase no registraron su edad y eran sistemáticamente más jóvenes, imputar con la mediana global introduciría un sesgo incorrecto.

4. ¿Imputar con un valor constante es siempre buena idea?

No. Para variables continuas como edad, poner un 0 o un -1 distorsiona la distribución real de los datos y la relación física de la variable.

5. ¿Podría la edad estar correlacionada con supervivencia?

Sí, fuertemente. Los niños tenían prioridad de salvamento.

6. Si imputas edad con la media global, ¿estás perdiendo información por clase social?

Sí. Probablemente, la edad media en 1ª clase gente más rica y mayor era distinta a la de 3ª clase inmigrantes jóvenes. Usar un único valor global elimina esa distinción.

7. ¿Sería mejor imputar edad por grupo (por ejemplo, por clase del billete)?

Sí, sería mucho más preciso imputar la edad usando la mediana agrupada por pclass o por sex.

2.3 Codificación de variables categóricas

```
Dimensiones X_train: (623, 9)
Dimensiones X_test: (268, 9)
```

1. ¿Por qué necesitamos convertir variables categóricas a numéricas?

Los modelos de Machine Learning son algoritmos matemáticos que operan con ecuaciones. No pueden procesar texto como male o Southampton, necesitan representaciones numéricas 0 y 1.

2. ¿Qué significa drop_first=True?

Significa que al crear variables dummy One-Hot Encoding, eliminamos la primera columna resultante.

Ejemplo: Para Sex, en lugar de tener Sex_female, Sex_male, nos quedamos solo con Sex_male.

3. ¿Qué problema evita?

Evita la multicolinealidad perfecta o trampa de las variables ficticias. Si sabemos que alguien no es mujer, obligatoriamente es hombre en este dataset binario. Tener ambas columnas introduce información redundante que afecta a modelos lineales.

4. ¿Qué pasaría si no lo usamos?

Aumenta la dimensionalidad del dataset innecesariamente y puede causar inestabilidad en modelos como la Regresión Logística o Lineal.

5. ¿El árbol necesita necesariamente variables normalizadas?

No. Los árboles de decisión funcionan dividiendo los datos basándose en reglas de umbral (por ejemplo, "¿Es la Edad < 30?"). Al algoritmo solo le importa el orden de los datos, no su magnitud o distancia. Si normalizamos los datos, el valor numérico cambia, pero el orden se mantiene, por lo que el árbol haría exactamente el mismo corte y la estructura sería idéntica.

6. ¿Es necesario escalar variables para modelos basados en árboles?

No. Al igual que ocurre con un árbol individual, los modelos de ensemble como Random Forest o Boosting procesan cada variable de forma independiente para encontrar el mejor punto de corte. A diferencia de algoritmos como KNN o Redes Neuronales, que calculan distancias euclídeas (y donde una variable con números grandes dominaría a una de números pequeños), a los árboles no les afecta la escala de las variables.

7. ¿Por qué boosting y random forest no necesitan escalado?

Porque los árboles toman decisiones basándose en reglas de corte (ej: "Si Edad > 10, ir a la izquierda"). El resultado de la división es el mismo tanto si la edad es 10 como si está escalada a 0.5; el orden relativo de los datos no cambia.

PARTE 3 – Entrenamiento de modelos

3.1 Árbol de decisión

```
=== 3.1 ÁRBOL DE DECISIÓN ===
--- Árbol (Depth=3) ---
Accuracy: 0.7948 | Recall: 0.5437
Matriz de Confusión:
[[157  8]
 [ 47 56]]
-----
--- Árbol (Depth=5) ---
Accuracy: 0.7873 | Recall: 0.5922
Matriz de Confusión:
[[150 15]
 [ 42 61]]
-----
--- Árbol (Depth=20) ---
Accuracy: 0.7649 | Recall: 0.6311
Matriz de Confusión:
[[140 25]
 [ 38 65]]
-----
```

1. ¿Qué accuracy obtiene?

Obtiene 0.7948 con profundidad 3.

2. ¿Hay sobreajuste?

Sí. Al aumentar la profundidad a 20, el accuracy baja a 0.7649. El modelo memorizó los datos de entrenamiento (overfitting) y falló al generalizar en los datos de test.

3. ¿Qué ocurre si limitas la profundidad?

Al probar profundidades (3, 5, 20), observamos que limitar la profundidad podar el árbol a 3 mejora el resultado en test. Evita que el árbol aprenda el ruido de los datos.

3.2 Random Forest

```
=== 3.2 RANDOM FOREST ===
--- Random Forest (n=10) ---
Accuracy: 0.7985 | Recall: 0.6408
Matriz de Confusión:
[[148  17]
 [ 37  66]]
-----
--- Random Forest (n=50) ---
Accuracy: 0.7948 | Recall: 0.6699
Matriz de Confusión:
[[144  21]
 [ 34  69]]
-----
--- Random Forest (n=100) ---
Accuracy: 0.8060 | Recall: 0.6699
Matriz de Confusión:
[[147  18]
 [ 34  69]]
-----
```

Probar distintos n_estimators:

Se probaron 10, 50 y 100 estimadores.

1. ¿Mejora respecto al árbol?

Sí. Con n=100, el accuracy sube a 0.8060, superando al árbol individual (0.7948).

2. ¿Cuándo se estabiliza?

Se estabiliza a partir de los 50 estimadores. La diferencia entre 50 y 100 árboles es pequeña, indicando que añadir más árboles tiene rendimientos decrecientes.

3.3 Bagging

```
=== 3.3 BAGGING ===
--- Bagging ---
Accuracy: 0.7985 | Recall: 0.6699
Matriz de Confusión:
[[145  20]
 [ 34  69]]
-----
```

1. ¿Se parece más al árbol o al Random Forest?

Se parece más al Random Forest (Accuracy: 0.7985). Porque el Random Forest es, en esencia, una versión especializada de Bagging aplicado a árboles de decisión. Ambos utilizan el principio de entrenar múltiples modelos con subconjuntos aleatorios de datos y promediar sus resultados para reducir la varianza.

3.4 AdaBoost

```
=== 3.4 ADABOOST ===
--- AdaBoost ---
Accuracy: 0.7948 | Recall: 0.6990
Matriz de Confusión:
[[141  24]
 [ 31  72]]
-----
```

1. ¿Reduce el sesgo?

Sí. AdaBoost entrena modelos secuencialmente, donde cada nuevo modelo se centra en corregir los errores sesgo de los anteriores.

2. ¿Mejora recall?

Sí. Obtiene un Recall de 0.6990, que es de los más altos de la comparativa, indicando que es muy bueno identificando los casos positivos los supervivientes.

3.5 Gradient Boosting

```
=== 3.5 GRADIENT BOOSTING ===
--- Gradient Boosting (LR=0.01) ---
Accuracy: 0.8022 | Recall: 0.5534
Matriz de Confusión:
[[158   7]
 [ 46  57]]
-----
--- Gradient Boosting (LR=0.1) ---
Accuracy: 0.8097 | Recall: 0.6311
Matriz de Confusión:
[[152  13]
 [ 38  65]]
-----
--- Gradient Boosting (LR=1.0) ---
Accuracy: 0.7985 | Recall: 0.6505
Matriz de Confusión:
[[147  18]
 [ 36  67]]
-----
```

Cambiar learning_rate:

Se probaron tasas de 0.01, 0.1 y 1.0.

1. ¿Qué ocurre con learning rate alto (1.0)?

El accuracy baja a 0.7985. El modelo da pasos demasiado grandes al aprender, oscila y no logra converger en el mínimo óptimo.

2. ¿Y con uno bajo (0.01)?

El accuracy es de 0.8022. Es estable, pero aprende muy despacio y podría necesitar muchos más árboles para alcanzar su máximo potencial (riesgo de underfitting si los árboles son pocos).

El mejor resultado fue con LR=0.1 (0.8097).

3.6 XGBoost

```
=== 3.6 XGBOOST ===
--- XGBoost ---
Accuracy: 0.7910 | Recall: 0.6602
Matriz de Confusión:
[[144  21]
 [ 35  68]]
-----
```

1. ¿Obtiene el mejor resultado?

No en esta ejecución específica (Accuracy: 0.7910). El Gradient Boosting estándar obtuvo un resultado ligeramente superior (0.8097).

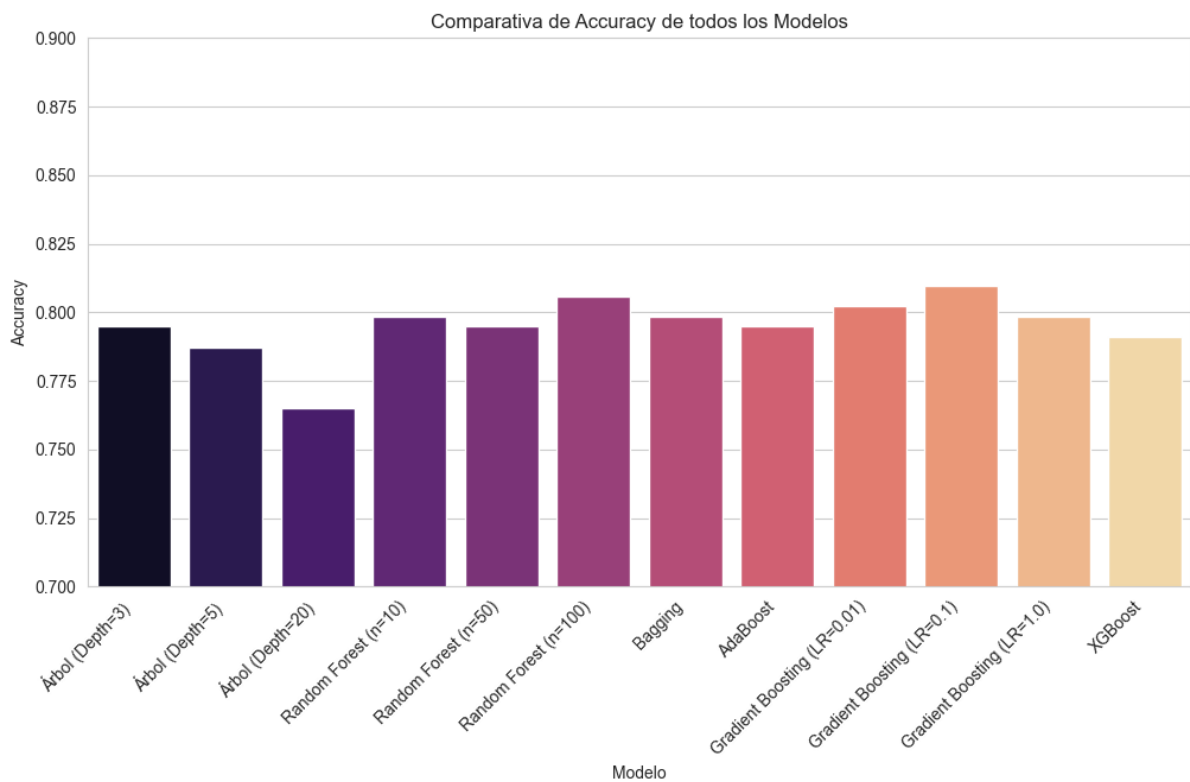
2. ¿La mejora es grande o marginal?

La diferencia es marginal y, en este caso, negativa. XGBoost es muy potente pero a veces requiere un ajuste de hiperparámetros más fino que los valores por defecto para superar a otros modelos.

PARTE 4 – Comparación Global

Modelo	Accuracy	Precision	Recall	F1	Interpretabilidad	Coste
Gradient Boosting (LR=0.1)	0.8097	0.8333	0.6311	0.7182	Baja (Caja negra)	Alto
Random Forest (n=100)	0.8060	0.7931	0.6699	0.7263	Media	Medio/Alto
Gradient Boosting (LR=0.01)	0.8022	0.8906	0.5534	0.6826	Baja	Alto
Random Forest (n=10)	0.7985	0.7952	0.6408	0.7097	Media	Bajo

Gradient Boosting (LR=1.0)	0.7985	0.7882	0.6505	0.7128	Baja	Alto
Bagging	0.7985	0.7753	0.6699	0.7188	Media	Medio
Árbol (Depth=3)	0.7948	0.8750	0.5437	0.6707	Alta (Muy clara)	Muy Bajo
Random Forest (n=50)	0.7948	0.7667	0.6699	0.7150	Media	Medio
AdaBoost	0.7948	0.7500	0.6990	0.7236	Baja	Medio
XGBoost	0.7910	0.7640	0.6602	0.7083	Baja	Alto
Árbol (Depth=5)	0.7873	0.8026	0.5922	0.6816	Media/Alta	Bajo
Árbol (Depth=20)	0.7649	0.7222	0.6311	0.6736	Baja (Complejo)	Bajo



1. ¿Qué variables son más importantes?

El sexo (Sex_male) y la clase (Pclass) son las más determinantes, seguidas de la edad.

2. ¿Es el sexo determinante?

Absolutamente. Ser mujer aumentaba drásticamente la probabilidad de predecir supervivencia debido al protocolo de evacuación.

3. ¿Influye la clase del billete?

Sí. Los pasajeros de 1ª clase tenían camarotes más cerca de la cubierta y prioridad, lo que el modelo refleja con una alta importancia de esta variable.

4. ¿Hay sesgo social implícito?

Sí. El modelo ha aprendido el sesgo clasista y de género de la sociedad de 1912.

Si este modelo se usara para priorizar evacuación:

5. ¿Sería ético?

No. Usar este modelo hoy sería discriminatorio, ya que priorizaría salvar a los ricos (1ª clase) sobre los pobres, lo cual es éticamente inaceptable en la actualidad.

6. ¿El modelo refleja decisiones humanas pasadas?

Sí, el modelo actúa como un espejo de las decisiones humanas tomadas aquella noche.

7. ¿Está aprendiendo patrones sociales históricos?

Sí, está perpetuando matemáticamente los patrones de comportamiento y prejuicios de la época.

8. Ordena modelos de más interpretable a menos.

Árbol de Decisión > Bagging/Random Forest > AdaBoost > Gradient Boosting/XGBoost.

9. ¿Qué modelo pondrías en producción?

Elegiría el Random Forest (n=100).

10. ¿Elegirías el más preciso?

No necesariamente solo por la precisión numérica pura (donde ganó Gradient Boosting por muy poco, 0.003).

11. ¿Cómo justificarías tu decisión?

Justificaría el Random Forest porque es mucho más estable y fácil de entrenar. Tiene un mejor equilibrio de métricas (Accuracy 80.6% y un Recall superior al de Gradient Boosting: 0.67 vs 0.63).

En un problema de rescate, el Recall es crítico, y Random Forest lo hace mejor que el modelo más preciso.

¿Qué parte ha costado más y cuál ha sido la más interesante?

La parte que más nos ha costado ha sido entender el ajuste fino de los hiperparámetros en los modelos de Boosting como el learning_rate, ya que un cambio pequeño afectaba mucho al resultado. Lo más interesante ha sido comprobar empíricamente cómo la unión de muchos modelos como en Random Forest supera consistentemente a un solo modelo complejo.

